

Software Requirements Specification

Bohloko Family Farm Integrated Poultry Management System

Version: 1.0

Date: August 2026

Prepared by: Automated Codebase Analysis

Status: Active Development

Software Requirements Specification (SRS)

Bohloko Family Farm — Integrated Poultry Management System

Version: 1.0
Date: August 2026
Prepared by: Automated Codebase Analysis
Project Repository: BohlokoFamilyFarm
Status: Active Development

Table of Contents

- 1. [Introduction](#)
- 2. [Overall Description](#)
- 3. [Functional Requirements](#)
- 4. [Non-Functional Requirements](#)
- 5. [System Architecture](#)
- 6. [Data Requirements](#)
- 7. [User Interface Requirements](#)
- 8. [External Interface Requirements](#)
- 9. [System Features — Detailed Specifications](#)
- 10. [Appendices](#)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a complete, line-by-line analysis of the Bohloko Family Farm poultry management system. It describes all functional and non-functional requirements, system architecture, data models, API interfaces, and user interface specifications derived from exhaustive analysis of the project source code, documentation, configuration files, and HTML pages.

1.2 Scope

The Bohloko Family Farm system is a full-stack web application designed to manage the complete lifecycle of a poultry farming operation — from bird procurement and production through processing, inventory, sales, and delivery. The system serves multiple user roles including farm managers, poultry attendants, processing staff, sales assistants, and customers.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
JWT	JSON Web Token
RBAC	Role-Based Access Control
CRUD	Create, Read, Update, Delete

Term	Definition
PWA	Progressive Web App
i18n	Internationalization
FIFO	First In, First Out
COGS	Cost of Goods Sold
P&L	Profit and Loss
POPIA	Protection of Personal Information Act (South Africa)
ZAR	South African Rand
SPA	Single Page Application
API	Application Programming Interface
ORM	Object-Relational Mapping

1.4 References

Document	Location
Requirements Document	Documentation/Requirements_Document.md
Use Case Document	Documentation/UseCase_Document.md
Entity Relationship Diagram	Documentation/Entity_Relationship_Diagram.md
Data Schema	Documentation/DATA_SCHEMA.md
Class Diagrams	Documentation/Class_Diagrams.md
Authentication Implementation	Documentation/AUTHENTICATION_IMPLEMENTATION.md
Backend Services & Routes Summary	Documentation/BACKEND_SERVICES_ROUTES_SUMMARY.md
SOLID Principles Implementation	Documentation/SOLID_PRINCIPLES_IMPLEMENTATION_SUMMARY.md
Frontend Backend Mapping	Documentation/FRONTEND_BACKEND_MAPPING.md
Render Deployment	Documentation/RENDER_DEPLOYMENT.md
Backup & Recovery Procedures	Documentation/BACKUP_RECOVERY_PROCEDURES.md

2. Overall Description

2.1 Product Perspective

The Bohloko Family Farm system is a standalone, full-stack web application comprising:

- **Frontend:** Static HTML5/CSS3/JavaScript Progressive Web App (PWA)
- **Backend:** Node.js/Express.js RESTful API server
- **Database:** PostgreSQL (production) / MySQL (development) / NeDB (fallback)
- **Deployment:** Render.com (backend) + GitHub Pages (frontend)



2.2 Product Functions (High-Level Summary)

Function Category	Description
Authentication & Authorization	JWT-based login, role-based access control, password reset
User Management	CRUD operations, role assignment, status management
Product Catalog	Product listings, tiered pricing, categories, featured items
Shopping Cart & Checkout	Persistent cart, delivery options, checkout workflow
Order Management	Order creation, status tracking, cancellation
Payment Processing	Cash/Card/EFT/Mobile, receipts, refunds
Inventory Management	Stock tracking, FIFO picking, transfers, low-stock alerts
Harvest & Processing	Live bird → processed product pipeline, yield tracking
Production Cycle Management	Breeding cycles, daily logs, mortality tracking
Poultry Care	Daily care logs, health checks, vaccinations, feeding
Medication Tracking	Medication records, withdrawal periods, compliance
Quality Control	Inspections, corrective actions, certificates
Customer Relationship Management	Customer profiles, loyalty, feedback, campaigns
Analytics & Reporting	Sales, inventory, production, P&L analytics
Compliance Management	Food safety, Halal, organic compliance tracking
Notification System	Configurable alerts, multi-channel delivery
Data Management	Backup, restore, export, archiving
System Configuration	Configurable business rules, thresholds, pricing
Internationalization	English and Sesotho language support
Progressive Web App	Offline support, installable, background sync

2.3 User Classes and Characteristics

Role	Access Level	Key Responsibilities
Admin / Farm Manager	Full system access	User management, system configuration, analytics, compliance, data backup
Poultry Attendant	Production data only	Daily bird care, health checks, vaccinations, feeding logs, mortality tracking
Processing Staff	Processing data only	Quality checks, harvest processing, yield recording, packaging
Sales Assistant	Sales data only	Order management, payment processing, dispatch, picking lists
Customer	Own data only	Browse products, place orders, track orders, view loyalty

2.4 Operating Environment

Component	Specification
Server Runtime	Node.js 18+
Web Framework	Express.js 4.18+
Database	PostgreSQL 14+ (production), MySQL 8+ (development)
Frontend	HTML5, CSS3, Bootstrap 5.3, Vanilla JavaScript ES6+
Browser Support	Chrome, Firefox, Safari, Edge (latest 2 versions)
Mobile	Responsive design, PWA installable on Android/iOS
Deployment	Render.com (backend), GitHub Pages (frontend)

2.5 Design and Implementation Constraints

Constraint	Details
South African Regulations	Must comply with POPIA for data protection, SA food safety regulations
Currency	All monetary values in South African Rand (ZAR)
Languages	English and Sesotho (South African language)
Hosting Budget	Render.com free tier, GitHub Pages (free)
Database	Free PostgreSQL tier on Render (limited connections/storage)
No TypeScript	Codebase uses vanilla JavaScript
PWA Offline	Must function with limited connectivity (farm environments)

2.6 Assumptions and Dependencies

Assumptions:

- Users have access to modern web browsers
- Internet connectivity is available (intermittent in rural areas)
- Farm has at least one device with browser access
- Admin can create user accounts for staff

Dependencies:

- Render.com hosting availability
- PostgreSQL database availability on Render
- Gmail API for Google OAuth (optional)
- PayFast/Yoco for payment processing (planned)
- Twilio/Africa's Talking for SMS notifications (planned)

3. Functional Requirements

3.1 Authentication & Authorization (FR-AUTH)

ID	Requirement	Priority	Status
FR-AUTH-001	User registration with email, password, role	High	Implemented
FR-AUTH-002	User login with email/password, JWT token issued	High	Implemented
FR-AUTH-003	Google OAuth 2.0 social login (optional)	Medium	Implemented
FR-AUTH-004	Password reset via email token (1hr expiry)	High	Implemented
FR-AUTH-005	Password complexity enforcement (8+ chars, upper, lower, number, special)	High	Implemented
FR-AUTH-006	Account lockout after 5 failed attempts (30min duration)	High	Implemented
FR-AUTH-007	JWT token with 24hr expiry	High	Implemented
FR-AUTH-008	Token blacklist for logout	High	Implemented
FR-AUTH-009	30-minute session timeout with auto-logout	Medium	Implemented
FR-AUTH-010	Role-based access control (5 roles)	High	Implemented
FR-AUTH-011	Password visibility toggle on login forms	Low	Implemented
FR-AUTH-012	"Remember Me" functionality	Low	Partial

3.2 User Management (FR-USER)

ID	Requirement	Priority	Status
FR-USER-001	Admin can create new user accounts	High	Implemented
FR-USER-002	Admin can view all users with filtering	High	Implemented
FR-USER-003	Admin can update user roles	High	Implemented
FR-USER-004	Admin can enable/disable user accounts	High	Implemented
FR-USER-005	Admin can soft-delete users	Medium	Implemented
FR-USER-006	Users can update own profile (name, phone)	Medium	Implemented

ID	Requirement	Priority	Status
FR-USER-007	User search and filter by status/type	Medium	Implemented
FR-USER-008	Bulk user operations (approve/suspend/reject)	Medium	Implemented
FR-USER-009	Only Farm Manager and Poultry Attendant can self-register	High	Implemented
FR-USER-010	User approval workflow for new registrations	Medium	Partial

3.3 Product Management (FR-PROD)

ID	Requirement	Priority	Status
FR-PROD-001	Admin can create, update, delete products	High	Implemented
FR-PROD-002	Product categories: Live Birds, Processed, Value-Added, By-Products	High	Implemented
FR-PROD-003	Tiered pricing: Consumer, Restaurant, Retailer, Distributor	High	Implemented
FR-PROD-004	Products can be marked as featured	Medium	Implemented
FR-PROD-005	Product image support	Medium	Implemented
FR-PROD-006	Product search and filtering	Medium	Implemented
FR-PROD-007	Stock quantity tracking per product	High	Implemented
FR-PROD-008	Low stock threshold configuration per product	Medium	Implemented
FR-PROD-009	Product SKU management	Medium	Implemented
FR-PROD-010	Halal/Organic certification flags	Low	Implemented
FR-PROD-011	Shelf life tracking (days)	Medium	Implemented

3.4 Shopping Cart & Checkout (FR-CART)

ID	Requirement	Priority	Status
FR-CART-001	Persistent shopping cart (localStorage + server)	High	Implemented
FR-CART-002	Add/remove/update items in cart	High	Implemented
FR-CART-003	Quantity controls (+/- buttons)	High	Implemented
FR-CART-004	VAT calculation (configurable rate)	Medium	Implemented
FR-CART-005	Bulk discount display	Medium	Implemented
FR-CART-006	Delivery options: Pickup, Farm Gate, Local Delivery	Medium	Implemented
FR-CART-007	Checkout form: Name, Email, Phone, Address, Payment Method	High	Implemented
FR-CART-008	Payment methods: Cash, Bank Transfer, Mobile Money	Medium	Implemented
FR-CART-009	Free delivery threshold configuration	Low	Implemented
FR-CART-010	Server-side cart sync when logged in	Medium	Implemented

3.5 Order Management (FR-ORDER)

ID	Requirement	Priority	Status
FR-ORDER-001	Order creation from cart	High	Implemented
FR-ORDER-002	Order status tracking: pending → confirmed → preparing → ready → delivered	High	Implemented

ID	Requirement	Priority	Status
FR-ORDER-003	Customer can view own order history	High	Implemented
FR-ORDER-004	Staff can view all orders with filtering	High	Implemented
FR-ORDER-005	Staff can update order status	High	Implemented
FR-ORDER-006	Customer can cancel own orders	Medium	Implemented
FR-ORDER-007	Order number generation	Medium	Implemented
FR-ORDER-008	Order notes/special instructions	Low	Implemented
FR-ORDER-009	Delivery address management	Medium	Implemented
FR-ORDER-010	Product snapshots in order line items (audit trail)	Medium	Implemented
FR-ORDER-011	Invoice generation per order	Low	Implemented

3.6 Payment Processing (FR-PAY)

ID	Requirement	Priority	Status
FR-PAY-001	Record payment against order	High	Implemented
FR-PAY-002	Payment methods: Cash, Card, EFT, Mobile	Medium	Implemented
FR-PAY-003	Payment status tracking: pending, completed, refunded	High	Implemented
FR-PAY-004	Process refund	Medium	Implemented
FR-PAY-005	Revenue statistics and reporting	Medium	Implemented
FR-PAY-006	Payment reference/transaction tracking	Medium	Implemented
FR-PAY-007	Partial payment support	Low	Partial
FR-PAY-008	PayFast integration (test mode)	Medium	Planned
FR-PAY-009	Automatic receipt generation (PDF)	Low	Planned

3.7 Inventory Management (FR-INV)

ID	Requirement	Priority	Status
FR-INV-001	Create and manage inventory batches	High	Implemented
FR-INV-002	Real-time stock level tracking	High	Implemented
FR-INV-003	Low stock alerts with configurable thresholds	High	Implemented
FR-INV-004	Stock adjustments (damaged, expired, transferred)	High	Implemented
FR-INV-005	Inter-location transfer tracking	Medium	Implemented
FR-INV-006	FIFO (First In, First Out) picking for orders	Medium	Implemented
FR-INV-007	Expiry date tracking	High	Implemented
FR-INV-008	Batch number management	Medium	Implemented
FR-INV-009	Picking list generation for orders	Medium	Implemented
FR-INV-010	Inventory report generation	Medium	Implemented
FR-INV-011	Stock reservation on order placement	Medium	Implemented
FR-INV-012	Transfer audit trail	Low	Implemented

ID	Requirement	Priority	Status
FR-INV-013	Storage location tracking	Low	Implemented

3.8 Harvest & Processing (FR-HARV)

ID	Requirement	Priority	Status
FR-HARV-001	Create harvest batches linked to production cycles	High	Implemented
FR-HARV-002	Record birds harvested, live weight, harvest date	High	Implemented
FR-HARV-003	Create processing batches per product type	High	Implemented
FR-HARV-004	Track processing steps: Slaughter, Pluck, Eviscerate, Cut, Portion, Package, Label, Freeze, Store	High	Implemented
FR-HARV-005	Record yield: input weight, output weight, waste weight	High	Implemented
FR-HARV-006	Calculate yield percentages	Medium	Implemented
FR-HARV-007	Quality checks during processing	High	Implemented
FR-HARV-008	Shift tracking (Morning/Afternoon/Night)	Low	Implemented
FR-HARV-009	Auto-create inventory batches on harvest completion	High	Implemented
FR-HARV-010	Harvest statistics and dashboard	Medium	Implemented
FR-HARV-011	Staff assignment to processing batches	Low	Implemented

3.9 Production Cycle Management (FR-PROD)

ID	Requirement	Priority	Status
FR-PROD-001	Create production cycles (breeding batches)	High	Implemented
FR-PROD-002	Cycle status: planning → active → harvesting → completed	High	Implemented
FR-PROD-003	Daily log entries: bird count, feed, water, mortality	High	Implemented
FR-PROD-004	Mortality tracking and rate calculation	High	Implemented
FR-PROD-005	Health check records per cycle	Medium	Implemented
FR-PROD-006	Vaccination schedule management	Medium	Implemented
FR-PROD-007	Weight tracking over time	Medium	Implemented
FR-PROD-008	Feed records (type, quantity, cost)	Medium	Implemented
FR-PROD-009	Environment records (temperature, humidity, ventilation, lighting)	Medium	Implemented
FR-PROD-010	Production cycle dashboard	Medium	Implemented
FR-PROD-011	Expected vs actual bird count tracking	Low	Implemented

3.10 Poultry Care Operations (FR-CARE)

ID	Requirement	Priority	Status
FR-CARE-001	Daily care log recording	High	Implemented
FR-CARE-002	Health check recording with severity levels	High	Implemented
FR-CARE-003	Medication logging with withdrawal periods	High	Implemented
FR-CARE-004	Weight recording (average weight, sample size)	Medium	Implemented

ID	Requirement	Priority	Status
FR-CARE-005	Feed logging (type: Starter/Grower/Finisher/Layer/Supplement)	Medium	Implemented
FR-CARE-006	Environment logging (temp, humidity, ventilation, lighting)	Medium	Implemented
FR-CARE-007	Upcoming vaccination alerts	Medium	Implemented
FR-CARE-008	Recent activity feed	Low	Implemented
FR-CARE-009	Mobile-first interface for field use	High	Implemented

3.11 Medication Tracking (FR-MED)

ID	Requirement	Priority	Status
FR-MED-001	Record medication administration per batch/cycle	High	Implemented
FR-MED-002	Medication types: Antibiotic, Vitamin, Vaccine, Supplement	High	Implemented
FR-MED-003	Withdrawal period tracking (blocks processing until safe)	High	Implemented
FR-MED-004	Active medication listing	Medium	Implemented
FR-MED-005	Expiring medication alerts	Medium	Implemented
FR-MED-006	Complete/cancel medication records	Medium	Implemented
FR-MED-007	Vaccination schedule with due/completed status	Medium	Implemented

3.12 Quality Control (FR-QC)

ID	Requirement	Priority	Status
FR-QC-001	Record quality checks at harvest, processing, packaging	High	Implemented
FR-QC-002	Check types: Temperature, Visual, Weight, Hygiene, Packaging, Label	High	Implemented
FR-QC-003	Results: Pass / Fail / Conditional	High	Implemented
FR-QC-004	Temperature recording (°C)	Medium	Implemented
FR-QC-005	Findings/notes documentation	Medium	Implemented
FR-QC-006	Quality check history per batch	Medium	Implemented
FR-QC-007	Corrective action documentation	Medium	Partial
FR-QC-008	Certificate of conformity generation	Low	Planned

3.13 Customer Relationship Management (FR-CRM)

ID	Requirement	Priority	Status
FR-CRM-001	Customer profile management	High	Implemented
FR-CRM-002	Customer types: Individual, Restaurant, Retailer, Distributor	Medium	Implemented
FR-CRM-003	Loyalty program with tier system (Bronze → Diamond)	Medium	Implemented
FR-CRM-004	Points earning (1 point per ZAR 10 spent, configurable)	Medium	Implemented
FR-CRM-005	Feedback collection with ratings	Medium	Implemented
FR-CRM-006	Feedback response tracking	Low	Implemented
FR-CRM-007	Campaign management (promotions)	Low	Implemented

ID	Requirement	Priority	Status
FR-CRM-008	Customer segmentation	Low	Implemented
FR-CRM-009	Customer data export (CSV)	Medium	Implemented
FR-CRM-010	CRM dashboard with statistics	Medium	Implemented

3.14 Analytics & Reporting (FR-ANAL)

ID	Requirement	Priority	Status
FR-ANAL-001	Production analytics (mortality, growth rates)	Medium	Implemented
FR-ANAL-002	Sales analytics (revenue, trends)	Medium	Implemented
FR-ANAL-003	Inventory analytics (turnover, ABC analysis)	Medium	Implemented
FR-ANAL-004	Profit & Loss reporting	Medium	Implemented
FR-ANAL-005	Dashboard statistics (orders, inventory, cycles)	High	Implemented
FR-ANAL-006	Data export: JSON/CSV for all entities	Medium	Implemented
FR-ANAL-007	Date-range filtering for analytics	Low	Implemented

3.15 Compliance Management (FR-COMP)

ID	Requirement	Priority	Status
FR-COMP-001	Quality check records	High	Implemented
FR-COMP-002	Compliance record tracking	Medium	Implemented
FR-COMP-003	Audit scheduling and history	Medium	Implemented
FR-COMP-004	Compliance report generation	Medium	Implemented
FR-COMP-005	Halal compliance tracking	Low	Implemented
FR-COMP-006	Food safety compliance	Medium	Partial

3.16 Notification System (FR-NOTIF)

ID	Requirement	Priority	Status
FR-NOTIF-001	In-app notification delivery	High	Implemented
FR-NOTIF-002	Unread notification count	Medium	Implemented
FR-NOTIF-003	Mark notifications as read	Medium	Implemented
FR-NOTIF-004	Delete notifications	Low	Implemented
FR-NOTIF-005	Notification configuration per type	Medium	Implemented
FR-NOTIF-006	Email notifications (planned)	Medium	Planned
FR-NOTIF-007	SMS notifications (planned)	Low	Planned

3.17 Data Management (FR-DATA)

ID	Requirement	Priority	Status
FR-DATA-001	Automated backup creation	High	Implemented

ID	Requirement	Priority	Status
FR-DATA-002	Backup restore functionality	High	Implemented
FR-DATA-003	Data export: JSON/CSV	Medium	Implemented
FR-DATA-004	System statistics dashboard	Medium	Implemented
FR-DATA-005	Data validation and integrity checks	Medium	Implemented
FR-DATA-006	Data archiving (90 days to 2 years)	Low	Implemented
FR-DATA-007	Database migration (NeDB → Firestore)	Low	Planned
FR-DATA-008	30-day backup retention	Medium	Implemented

3.18 System Configuration (FR-CONFIG)

ID	Requirement	Priority	Status
FR-CONFIG-001	Configurable VAT/Tax rate	Medium	Implemented
FR-CONFIG-002	Configurable delivery fees and thresholds	Medium	Implemented
FR-CONFIG-003	Bulk discount tier configuration	Low	Implemented
FR-CONFIG-004	Low stock threshold configuration	Medium	Implemented
FR-CONFIG-005	Currency configuration	Low	Implemented
FR-CONFIG-006	Session timeout configuration	Low	Implemented
FR-CONFIG-007	Max login attempts configuration	Low	Implemented
FR-CONFIG-008	Lock duration configuration	Low	Implemented

3.19 Internationalization (FR-I18N)

ID	Requirement	Priority	Status
FR-I18N-001	English language support	High	Implemented
FR-I18N-002	Sesotho language support	High	Implemented
FR-I18N-003	Runtime language switching	Medium	Implemented
FR-I18N-004	Language preference persistence (localStorage)	Medium	Implemented
FR-I18N-005	data-i18n attribute-based translation	Medium	Implemented
FR-I18N-006	~80% UI string coverage	Medium	Implemented

3.20 Progressive Web App (FR-PWA)

ID	Requirement	Priority	Status
FR-PWA-001	Service worker for offline caching	High	Implemented
FR-PWA-002	Stale-while-revalidate strategy	Medium	Implemented
FR-PWA-003	Offline fallback page	Medium	Implemented
FR-PWA-004	Background sync (IndexedDB)	Low	Implemented
FR-PWA-005	Installable (add to home screen)	Medium	Implemented
FR-PWA-006	Offline indicator	Low	Implemented

3.21 Employee Management (FR-EMP)

ID	Requirement	Priority	Status
FR-EMP-001	Employee listing and management	Medium	Implemented
FR-EMP-002	Department management	Low	Implemented
FR-EMP-003	Employee statistics	Low	Implemented
FR-EMP-004	Employee status toggle (active/inactive)	Low	Implemented

3.22 API Key Management (FR-APIKEY)

ID	Requirement	Priority	Status
FR-APIKEY-001	Generate API keys for external integrations	Low	Implemented
FR-APIKEY-002	Revoke API keys	Low	Implemented
FR-APIKEY-003	API key listing	Low	Implemented

3.23 System Logging (FR-LOG)

ID	Requirement	Priority	Status
FR-LOG-001	Structured system logging	Medium	Implemented
FR-LOG-002	Log query and filtering	Medium	Implemented
FR-LOG-003	Log cleanup/retention	Low	Implemented
FR-LOG-004	Log statistics	Low	Implemented

4. Non-Functional Requirements

4.1 Security (NFR-SEC)

ID	Requirement	Implementation
NFR-SEC-001	Password hashing with bcrypt (12 salt rounds)	bcryptjs library
NFR-SEC-002	JWT authentication with token expiry	jsonwebtoken (24hr expiry)
NFR-SEC-003	Token blacklist for logout	In-memory Set (resets on restart)
NFR-SEC-004	Rate limiting (100 req/15min per IP)	express-rate-limit
NFR-SEC-005	HTTP security headers	Helmet.js
NFR-SEC-006	CORS configuration	cors middleware
NFR-SEC-007	Input validation on all write endpoints	express-validator
NFR-SEC-008	XSS prevention (HTML escaping)	escapeHtml() function
NFR-SEC-009	SQL injection prevention	Knex parameterized queries
NFR-SEC-010	Account lockout (5 attempts → 30min lock)	UserService
NFR-SEC-011	Path traversal prevention (backup files)	Filename validation

ID	Requirement	Implementation
NFR-SEC-012	SameSite cookie policy	Cookie configuration
NFR-SEC-013	Environment variable secrets	dotenv

4.2 Performance (NFR-PERF)

ID	Requirement	Target
NFR-PERF-001	API response time	< 200ms average
NFR-PERF-002	Page load time	< 3 seconds
NFR-PERF-003	Database connection pooling	Min: 2, Max: 10 connections
NFR-PERF-004	Pagination on all list endpoints	Configurable page/limit
NFR-PERF-005	Service worker caching	Stale-while-revalidate
NFR-PERF-006	Static asset optimization	CDN-ready

4.3 Reliability (NFR-REL)

ID	Requirement	Implementation
NFR-REL-001	Automated daily backups	Cron job → /data/backups/
NFR-REL-002	Backup retention: 30 days	Rolling window
NFR-REL-003	Recovery Time Objective (RTO)	< 5 minutes
NFR-REL-004	Recovery Point Objective (RPO)	< 24 hours
NFR-REL-005	Health check endpoint	GET /api/health
NFR-REL-006	Startup diagnostics	DB connectivity check on boot

4.4 Scalability (NFR-SCAL)

ID	Requirement	Implementation
NFR-SCAL-001	Stateless server (horizontal scaling)	JWT-based auth
NFR-SCAL-002	Database connection pooling	Knex pool config
NFR-SCAL-003	Modular architecture	Service-Repository pattern
NFR-SCAL-004	CDN-ready static assets	GitHub Pages

4.5 Maintainability (NFR-MAINT)

ID	Requirement	Implementation
NFR-MAINT-001	MVC-inspired modular structure	backend/ directory layout
NFR-MAINT-002	Service layer separation	19 service files
NFR-MAINT-003	Active Record model pattern	11 model files
NFR-MAINT-004	SOLID principles applied	DI container, interfaces
NFR-MAINT-005	Centralized error handling	Express error middleware
NFR-MAINT-006	Input validation middleware	express-validator

ID	Requirement	Implementation
NFR-MAINT-007	JSDoc documentation on services	Service files

4.6 Usability (NFR-USE)

ID	Requirement	Implementation
NFR-USE-001	Responsive design (mobile/tablet/desktop)	Bootstrap 5.3
NFR-USE-002	Mobile-first interfaces for staff	Poultry, Processing, Sales pages
NFR-USE-003	Bilingual support (English/Sesotho)	data-i18n attributes
NFR-USE-004	Consistent navigation patterns	Sidebar + bottom nav
NFR-USE-005	ARIA labels for screen readers	HTML accessibility attributes
NFR-USE-006	Keyboard navigation support	Tab order, focus indicators
NFR-USE-007	WCAG color contrast compliance	CSS variables
NFR-USE-008	Touch-friendly UI elements	Large buttons, swipe nav

4.7 Compliance (NFR-COMP)

ID	Requirement	Implementation
NFR-COMP-001	POPIA compliance (South Africa)	Data protection measures
NFR-COMP-002	Food safety regulation compliance	Quality control workflows
NFR-COMP-003	Audit trail for critical operations	System logging

4.8 Portability (NFR-PORT)

ID	Requirement	Implementation
NFR-PORT-001	Browser-based access	No installation required
NFR-PORT-002	Cross-platform compatibility	Windows, macOS, Linux, Android, iOS
NFR-PORT-003	PWA installable	Add to home screen

5. System Architecture

5.1 Technology Stack

Layer	Technology	Version
Runtime	Node.js	18+
Web Framework	Express.js	4.18.2
ORM/Query Builder	Knex.js	3.3.0
Production Database	PostgreSQL	14+ (via pg driver 8.13.0)
Development Database	MySQL	8+ (via mysql2 3.23.2)
Fallback Database	NeDB	6.2.3 (file-based)

Layer	Technology	Version
Authentication	JWT (jsonwebtoken 9.0.2)	—
Password Hashing	bcryptjs	2.4.3
Validation	express-validator	7.0.1
Security Headers	Helmet	8.3.0
Rate Limiting	express-rate-limit	8.6.2
CORS	cors	2.8.5
Google OAuth	google-auth-library	11.0.0
Unique IDs	uuid	11.1.1
Frontend CSS	Bootstrap	5.3
Frontend Icons	Font Awesome	6.x
PDF Generation	pdfkit	0.19.1 (dev)
Testing	Jest	30.4.2 (dev)
Dev Server	nodemon	3.0.1 (dev)

5.2 Directory Structure

```
BohlokoFamilyFarm/
├── backend/
│   ├── config/
│   │   ├── db.js           # NeDB database connections
│   │   └── constants.js    # Role definitions, validation rules
│   ├── middleware/
│   │   ├── auth.js        # JWT verification, role authorization
│   │   └── validate.js    # express-validator chains
│   ├── models/            # Active Record pattern (11 models)
│   │   ├── User.js
│   │   ├── Order.js
│   │   ├── Inventory.js
│   │   ├── Production.js
│   │   ├── Cart.js
│   │   ├── Campaign.js
│   │   ├── CustomerProfile.js
│   │   ├── Feedback.js
│   │   ├── Loyalty.js
│   │   ├── PasswordReset.js
│   │   └── SystemConfig.js
│   ├── routes/            # API routes (22 files)
│   │   ├── auth.js
│   │   ├── users.js
│   │   ├── products.js
│   │   ├── orders.js
│   │   ├── payments.js
│   │   ├── cart.js
│   │   ├── inventory.js
│   │   ├── harvest.js
│   │   ├── production.js
│   │   ├── medication.js
│   │   ├── employees.js
│   │   ├── crm.js
│   │   ├── contact.js
│   │   ├── notifications.js
│   │   ├── notification-configs.js
│   │   ├── analytics.js
│   │   ├── config.js
│   │   ├── compliance.js
│   │   ├── data.js
│   │   ├── system-logs.js
│   │   ├── api-keys.js
│   │   └── health.js
│   └── services/          # Business logic (19 services)
│       ├── UserService.js
│       ├── ProductService.js
│       ├── OrderService.js
│       ├── PaymentService.js
│       ├── CartService.js
│       ├── InventoryService.js
│       ├── HarvestService.js
│       ├── ProductionService.js
│       ├── MedicationService.js
│       ├── EmployeeService.js
│       ├── CrmService.js
│       ├── ContactService.js
│       ├── NotificationService.js
│       ├── NotificationConfigService.js
│       ├── AnalyticsService.js
│       ├── ConfigService.js
│       ├── ComplianceService.js
│       ├── DataService.js
│       ├── SystemLogService.js
│       └── ApiKeyService.js
```

```

| | | | | seeds/                # Seed data
| | | | | migrations/         # Database migrations
| | | | | tests/              # Unit tests (Jest)
| | | | | knexfile.js         # Database configuration
| | | | | seed.js             # NeDB seed script
| | | | | seed-mysql.js       # MySQL seed script
| | | | | seed-pg.js          # PostgreSQL seed script
| | | | | seed-products.js    # Product seed script
| | | | | server.js           # Application entry point
| | | | | └─ package.json
| | | └─ assets/
| | | | └─ js/
| | | | | └─ api.js           # API client service
| | | | | └─ admin.js        # Admin dashboard logic
| | | | | └─ lang.js         # Language switching
| | | | └─ css/              # Stylesheets (8 files)
| | | | └─ images/           # Static images
| | └─ pages/
| | | └─ public/             # Public pages (7 HTML files)
| | | └─ dashboard/         # Customer dashboards (2 HTML files)
| | | └─ staff/             # Staff mobile pages (3 HTML files)
| | | └─ admin/             # Admin dashboard (1 SPA HTML)
| └─ translations/
| | └─ en.json              # English translations
| | └─ st.json              # Sesotho translations
└─ Documentation/          # Project documentation (48+ files)
└─ index.html              # Root redirect
└─ sw.js                   # Service worker
└─ render.yaml             # Render.com deployment config
└─ .github/                # GitHub Actions workflows

```

5.3 Architecture Patterns

Pattern	Implementation
Layered Architecture	Route → Service → Model → Database
Active Record	Models contain business logic and static query methods
Service Layer	19 service classes encapsulating business rules
Repository Pattern	Models act as repositories with CRUD methods
Facade Pattern	CrmService aggregates multiple CRM concerns
Middleware Chain	Auth → Validation → Route Handler → Error Handler
Dependency Injection	DI container for service wiring
DTO Pattern	Data transformation between layers
PWA Pattern	Service worker with stale-while-revalidate caching

5.4 SOLID Principles Implementation

Principle	Implementation
Single Responsibility	Each service handles one domain (UserService, OrderService, etc.)
Open/Closed	Services extensible via interfaces, not modified
Liskov Substitution	Model classes substitutable via DI container
Interface Segregation	Small, focused service interfaces
Dependency Inversion	Dependencies injected via container, not hard-coded

6. Data Requirements

6.1 Database Schema

User Table

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, UUID	Unique identifier
name	VARCHAR(100)	NOT NULL	Full name
email	VARCHAR(255)	UNIQUE, NOT NULL	Email address
password_hash	VARCHAR(255)	NOT NULL	bcrypt hashed password
role	ENUM	NOT NULL	Farm Manager, Poultry Attendant, Processing Staff, Sales Assistant, Customer
status	ENUM	DEFAULT 'active'	active, inactive, suspended, pending
phone	VARCHAR(20)	Phone number	
address	TEXT	Physical address	
department	VARCHAR(50)	Department assignment	
profile_image	VARCHAR(255)	Profile image URL	
last_login	TIMESTAMP	Last login timestamp	
failed_login_attempts	INT	DEFAULT 0	Failed login counter
lockout_until	TIMESTAMP	Account lock expiry	
created_at	TIMESTAMP	DEFAULT NOW	Creation timestamp
updated_at	TIMESTAMP	Last update timestamp	

Product Table

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, UUID	Unique identifier
name	VARCHAR(200)	NOT NULL	Product name
description	TEXT	Product description	
category	ENUM	NOT NULL	live_birds, processed, value_added, by_products
price	DECIMAL(10,2)	NOT NULL	Base price (ZAR)
unit	VARCHAR(20)	NOT NULL	kg, each, pack
image_url	VARCHAR(255)	Product image URL	
stock_quantity	INT	DEFAULT 0	Current stock level
low_stock_threshold	INT	DEFAULT 10	Low stock alert level
is_active	BOOLEAN	DEFAULT true	Product visibility
is_featured	BOOLEAN	DEFAULT false	Featured on homepage

Column	Type	Constraints	Description
display_order	INT	DEFAULT 0	Sort order
sku	VARCHAR(50)	UNIQUE	Stock keeping unit
shelf_life_days	INT	Days until expiry	Organic certification
is_organic	BOOLEAN	DEFAULT false	
is_halal	BOOLEAN	DEFAULT false	
price_tier_consumer	DECIMAL(10,2)	Consumer price	
price_tier_retailer	DECIMAL(10,2)	Retailer price	
price_tier_restaurant	DECIMAL(10,2)	Restaurant price	
price_tier_distributor	DECIMAL(10,2)	Distributor price	
min_stock_level	INT	DEFAULT 5	
created_at	TIMESTAMP	DEFAULT NOW	Minimum stock level
updated_at	TIMESTAMP		

Order Table

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, UUID	Unique identifier
customer_id	VARCHAR(36)	FK → User	Customer reference
customer_name	VARCHAR(100)	Denormalized customer name	
customer_email	VARCHAR(255)	Denormalized email	
customer_phone	VARCHAR(20)	Denormalized phone	
order_number	VARCHAR(20)	UNIQUE	
status	ENUM	NOT NULL	pending, confirmed, preparing, ready, delivered, completed, cancelled
payment_status	ENUM	DEFAULT 'pending'	pending, partial, paid, refunded
total_amount	DECIMAL(10,2)	NOT NULL	Order total (ZAR)
delivery_option	ENUM	pickup, farm_gate, local_delivery	
delivery_address	TEXT	Delivery address	
delivery_notes	TEXT	Delivery instructions	
special_instructions	TEXT	Special order notes	
notes	TEXT	Internal notes	
line_items	JSON	Array of {productId, name, price, quantity, snapshot}	
created_at	TIMESTAMP	DEFAULT NOW	
updated_at	TIMESTAMP		

InventoryBatch Table

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, UUID	Unique identifier
product_id	VARCHAR(36)	FK → Product	Product reference
batch_number	VARCHAR(50)	NOT NULL	Batch identifier
quantity	INT	NOT NULL	Quantity in stock
unit_cost	DECIMAL(10,2)	Cost per unit	
received_date	DATE	NOT NULL	Date received
expiry_date	DATE	Expiration date	
status	ENUM	DEFAULT 'active'	active, reserved, used, expired, damaged
storage_location	VARCHAR(100)	Storage location	
notes	TEXT	Additional notes	
created_at	TIMESTAMP	DEFAULT NOW	
updated_at	TIMESTAMP		

ProductionCycle Table

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, UUID	Unique identifier
cycle_number	VARCHAR(20)	UNIQUE	Human-readable cycle #
breed	VARCHAR(100)	NOT NULL	Chicken breed
batch_size	INT	NOT NULL	Number of birds
start_date	DATE	NOT NULL	Cycle start date
target_end_date	DATE	Expected end date	
current_count	INT	Current live bird count	
mortality_count	INT	DEFAULT 0	Cumulative mortality
status	ENUM	NOT NULL	planning, active, harvesting, completed, cancelled
notes	TEXT	Cycle notes	
created_by	VARCHAR(36)	FK → User	Created by user
created_at	TIMESTAMP	DEFAULT NOW	
updated_at	TIMESTAMP		

HarvestBatch Table

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, UUID	Unique identifier
cycle_id	VARCHAR(36)	FK → ProductionCycle	Production cycle
batch_number	VARCHAR(50)	UNIQUE	Batch identifier
birds_harvested	INT	NOT NULL	Number of birds

Column	Type	Constraints	Description
live_weight	DECIMAL(10,2)	Total live weight (kg)	Date of harvest
processing_date	DATE	NOT NULL	
shift	ENUM	morning, afternoon, night	
status	ENUM	DEFAULT 'pending'	
created_at	TIMESTAMP	DEFAULT NOW	

ProcessingBatch Table

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, UUID	Unique identifier
harvest_batch_id	VARCHAR(36)	FK → HarvestBatch	Harvest reference
product_type	VARCHAR(100)	NOT NULL	Product being produced
status	ENUM	DEFAULT 'in_progress'	in_progress, completed
output_quantity	INT	Final output quantity	Final output weight (kg)
output_weight	DECIMAL(10,2)	Final output weight (kg)	
waste_weight	DECIMAL(10,2)	Waste weight (kg)	
storage_location	VARCHAR(100)	Storage location	
created_at	TIMESTAMP	DEFAULT NOW	

Payment Table

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, UUID	Unique identifier
order_id	VARCHAR(36)	FK → Order	Order reference
method	ENUM	NOT NULL	cash, card, eft, mobile
amount	DECIMAL(10,2)	NOT NULL	Payment amount (ZAR)
status	ENUM	DEFAULT 'pending'	pending, completed, refunded
transaction_ref	VARCHAR(100)	Transaction reference	Processed by staff
processed_by	VARCHAR(36)	FK → User	
processed_date	TIMESTAMP	Processing timestamp	
created_at	TIMESTAMP	DEFAULT NOW	

CustomerProfile Table

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, UUID	Unique identifier
user_id	VARCHAR(36)	FK → User, UNIQUE	User reference
customer_type	ENUM	NOT NULL	individual, restaurant, retailer, distributor
loyalty_points	INT	DEFAULT 0	Current loyalty points

Column	Type	Constraints	Description
total_spent	DECIMAL(12,2)	DEFAULT 0	Lifetime spending
since	DATE	DEFAULT CURRENT_DATE	Customer since
tier	VARCHAR(20)	DEFAULT 'Bronze'	Loyalty tier
preferences	JSON	Customer preferences	
created_at	TIMESTAMP	DEFAULT NOW	

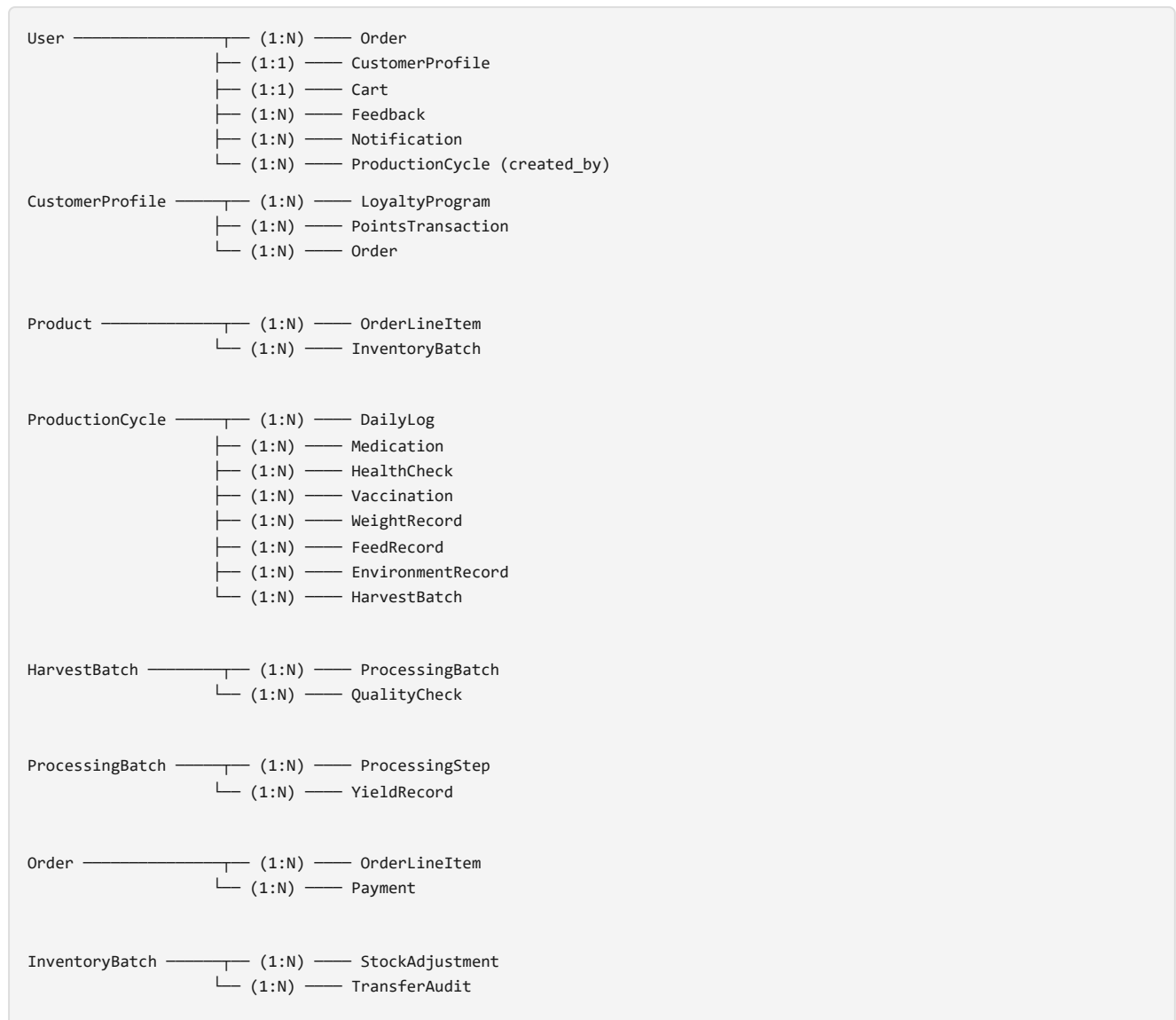
LoyaltyProgram Table

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, UUID	Unique identifier
name	VARCHAR(100)	NOT NULL	Program name
description	TEXT	Program description	
points_per_rand	INT	DEFAULT 1	Points per ZAR spent
discount_percentage	DECIMAL(5,2)	Discount percentage	
min_points	INT	Minimum points for tier	
max_discount	DECIMAL(10,2)	Maximum discount amount	
status	ENUM	DEFAULT 'active'	active, inactive
valid_from	DATE	Validity start	
valid_until	DATE	Validity end	

NotificationConfig Table

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, UUID	Unique identifier
type	VARCHAR(50)	NOT NULL	Notification type
name	VARCHAR(100)	NOT NULL	Config name
subject	VARCHAR(200)	Email subject	
message	TEXT	Message template	
channel	ENUM	DEFAULT 'in_app'	email, sms, push, in_app
enabled	BOOLEAN	DEFAULT true	Is enabled
frequency	VARCHAR(20)	DEFAULT 'immediate'	immediate, daily, weekly
conditions	JSON	Trigger conditions	

6.2 Entity Relationships



6.3 Data Backup & Recovery

Aspect	Specification
Backup Method	Automated cron job, JSON export
Backup Location	/data/backups/ with timestamped folders
Backup Frequency	Daily
Retention Policy	30 days rolling
Recovery Time Objective	< 5 minutes
Recovery Point Objective	< 24 hours
Backup Validation	Integrity checks on creation
Restore Process	Select backup → validate → overwrite → verify

7. User Interface Requirements

7.1 Page Inventory

Public Pages (7 pages)

Page	File	Purpose
Homepage	pages/public/index.html	Landing page, product showcase, marketing
About	pages/public/about.html	Farm story, mission, team
Shop	pages/public/shop.html	Product catalog, cart, checkout
Orders	pages/public/orders.html	Customer order history
Login	pages/public/login.html	User authentication
Signup	pages/public/signup.html	New account registration
Contact	pages/public/contact.html	Contact form, map, business hours
404	pages/public/404.html	Error page

Customer Dashboard Pages (2 pages)

Page	File	Purpose
Customer Dashboard	pages/dashboard/customer.html	Customer hub (orders, loyalty, products)
Restaurant Portal	pages/dashboard/restaurant.html	Wholesale customer dashboard

Staff Mobile Pages (3 pages)

Page	File	Purpose
Poultry Care	pages/staff/poultry.html	Daily bird care (mobile-first)
Processing	pages/staff/processing.html	Harvest & processing (mobile-first)
Sales	pages/staff/sales.html	Orders & payments (mobile-first)

Admin Dashboard (1 SPA with 13 sub-pages)

Sub-page	Purpose
Dashboard	Overview stats, recent orders, low stock alerts
Orders	Full order management with status updates
Inventory	Batch management, stock levels, transfers
Production	Cycle management, daily logs, mortality
Poultry Care	Health checks, medications, weight, feed, environment
Vaccinations	Vaccine schedule management
Harvest & Processing	Harvest batches, processing steps, yield
Users	User management, approval, role assignment
Compliance	Quality checks, compliance records

Sub-page	Purpose
Analytics	Sales, inventory, production analytics
Settings	System configuration (VAT, delivery, discounts)
Data Export	Backup, restore, export (JSON/CSV)
CRM	Customer profiles, loyalty, feedback, campaigns

7.2 Forms Summary

Page	Form	Fields
Login	loginForm	email, password, rememberMe
Forgot Password	Modal (3-step)	email → token + newPassword + confirmPassword
Signup	signupForm	firstName, lastName, email, phone, customerType, password, confirmPassword, agreeTerms
Shop Checkout	Modal	name, email, phone, deliveryOption, street, city, postalCode, paymentMethod
Contact	contactForm	name, email, phone, customerType, subject, message
Customer Settings	Modal	firstName, lastName, email(disabled), phone
Poultry Daily Log	Modal	cycle, date, birdCount, feed, water, mortality, activities
Poultry Health Check	Modal	cycle, date, birdsChecked, health, issues, actions
Poultry Medication	Modal	cycle, date, medName, dosage, reason, withdrawal
Poultry Weight	Modal	cycle, date, avgWeight, sampleSize, notes
Poultry Feed	Modal	cycle, date, feedType, feedQty, feedCost
Poultry Environment	Modal	cycle, date, temp, humidity, ventilation, lighting, notes
Processing Harvest	Modal	cycleId, productType, birds, liveWeight, harvestDate, shift, notes
Processing Step	Modal	batch, stepType, startTime, birdsProcessed, notes
Processing QC	Modal	batch, checkType, result, temperature, findings
Processing Yield	Modal	batch, productType, inputWeight, outputWeight, wasteWeight
Processing Complete	Modal	batch, outputQty, outputWeight, wasteWeight, storage, createInv
Sales Payment	Modal	order, payMethod, payAmount, payRef
Sales Dispatch	Modal	order, dispatchStatus, dispatchNote
Admin Settings	Forms	taxRate, shippingLocal, shippingThreshold, bulkTiers, lowStock, currency, sessionTimeout, maxLogin, lockDuration

7.3 Navigation Structure

PUBLIC (no auth required)	
└─	Homepage → About → Shop → Contact
└─	Login → (role-based redirect)
└─	Signup → Login
AFTER LOGIN (role-based routing):	
└─	Customer → pages/dashboard/customer.html
└─	Restaurant → pages/dashboard/restaurant.html
└─	Poultry Attendant → pages/staff/poultry.html
└─	Processing Staff → pages/staff/processing.html
└─	Sales Assistant → pages/staff/sales.html
└─	Farm Manager → pages/admin/dashboard.html
STAFF BOTTOM NAV (all staff pages):	
└─	Care (poultry.html)
└─	Process (processing.html)
└─	Sales (sales.html)
└─	Admin (dashboard.html)
ADMIN SIDEBAR (dashboard.html):	
└─	Dashboard
└─	Orders
└─	Inventory
└─	Production
└─	Poultry Care
└─	Vaccinations
└─	Harvest & Processing
└─	Users
└─	Compliance
└─	Analytics
└─	Settings
└─	Data Export
└─	CRM
└─	Logout
└─	View Website

7.4 Design System

Element	Specification
Primary Color	#1B4332 (Dark Green)
Accent Color	#D4A843 (Gold)
Background	#FDF8F0 (Cream)
CSS Framework	Bootstrap 5.3
Icons	Font Awesome 6.x
Fonts	System fonts (via Bootstrap)
Responsive Breakpoints	Mobile (< 768px), Tablet (768-1024px), Desktop (> 1024px)

8. External Interface Requirements

8.1 API Interface

The system exposes a RESTful API with the following characteristics:

Aspect	Specification
Base URL	Configurable via <code>API_BASE_URL</code> environment variable
Protocol	HTTP/HTTPS
Format	JSON request/response bodies
Authentication	Bearer token (JWT) in Authorization header
Rate Limit	100 requests per 15 minutes per IP
CORS	Enabled for frontend origin

8.2 Complete API Endpoint Inventory

Authentication (6 endpoints)

Method	Endpoint	Access	Description
POST	<code>/api/auth/register</code>	Public	Register new user
POST	<code>/api/auth/login</code>	Public	Login, returns JWT
POST	<code>/api/auth/logout</code>	Protected	Invalidate token
POST	<code>/api/auth/forgot-password</code>	Public	Request password reset
POST	<code>/api/auth/reset-password/:token</code>	Public	Reset password
GET	<code>/api/auth/me</code>	Protected	Get current user profile

User Management (6 endpoints)

Method	Endpoint	Access	Description
GET	<code>/api/users</code>	Farm Manager	List all users
GET	<code>/api/users/:id</code>	Farm Manager	Get user by ID
POST	<code>/api/users</code>	Farm Manager	Create new user
PUT	<code>/api/users/:id/role</code>	Farm Manager	Update user role
PUT	<code>/api/users/:id/status</code>	Farm Manager	Enable/disable user
DELETE	<code>/api/users/:id</code>	Farm Manager	Soft delete user

Products (9 endpoints)

Method	Endpoint	Access	Description
GET	<code>/api/products</code>	Public	List products (with filters)
GET	<code>/api/products/active</code>	Public	List active products
GET	<code>/api/products/featured</code>	Public	List featured products

Method	Endpoint	Access	Description
GET	/api/products/categories	Public	List product categories
GET	/api/products/:id	Public	Get product details
POST	/api/products	Admin	Create new product
PUT	/api/products/:id	Admin	Update product
DELETE	/api/products/:id	Admin	Soft delete product
POST	/api/products/:id/feature	Admin	Toggle featured status

Orders (6 endpoints)

Method	Endpoint	Access	Description
POST	/api/orders	Authenticated	Create new order
GET	/api/orders	Authenticated	List user's orders
GET	/api/orders/all	Staff	List all orders
GET	/api/orders/:id	Owner/Staff	Get order details
PUT	/api/orders/:id/status	Staff	Update order status
PUT	/api/orders/:id/cancel	Owner	Cancel order

Payments (6 endpoints)

Method	Endpoint	Access	Description
GET	/api/payments	Staff	List payments
GET	/api/payments/stats	Manager	Revenue statistics
GET	/api/payments/:id	Staff	Get payment details
POST	/api/payments	Staff	Create payment record
PUT	/api/payments/:id/process	Staff	Mark payment processed
PUT	/api/payments/:id/refund	Manager	Process refund

Cart (5 endpoints)

Method	Endpoint	Access	Description
GET	/api/cart	Authenticated	Get current cart
POST	/api/cart/items	Authenticated	Add item to cart
PUT	/api/cart/items/:productId	Authenticated	Update item quantity
DELETE	/api/cart/items/:productId	Authenticated	Remove item
DELETE	/api/cart	Authenticated	Clear cart

Inventory (10 endpoints)

Method	Endpoint	Access	Description
GET	/api/inventory	Staff	List inventory batches
GET	/api/inventory/low-stock	Staff	Low stock alerts
GET	/api/inventory/transfers	Staff	Transfer history
GET	/api/inventory/picking-list/:orderId	Staff	Generate picking list
GET	/api/inventory/report	Manager	Inventory report
POST	/api/inventory	Staff	Create inventory batch
PUT	/api/inventory/:id	Staff	Update batch
PUT	/api/inventory/:id/adjust	Staff	Adjust quantity
PUT	/api/inventory/:id/transfer	Staff	Transfer batch
DELETE	/api/inventory/:id	Staff	Remove batch

Harvest & Processing (15 endpoints)

Method	Endpoint	Access	Description
GET	/api/harvest/dashboard	Attendant	Harvest dashboard
GET	/api/harvest/batches	Attendant	List harvest batches
POST	/api/harvest/batches	Attendant	Create harvest batch
GET	/api/harvest/batches/:id	Attendant	Get batch details
PUT	/api/harvest/batches/:id	Attendant	Update batch
PUT	/api/harvest/batches/:id/complete	Attendant	Complete harvest
GET	/api/harvest/processing	Processing	List processing batches
POST	/api/harvest/processing	Processing	Start processing
PUT	/api/harvest/processing/:id/step	Processing	Update processing step
PUT	/api/harvest/processing/:id/complete	Processing	Complete processing
POST	/api/harvest/processing/:id/yield	Processing	Record yield
GET	/api/harvest/yield	Processing	Yield history
POST	/api/harvest/quality	Processing	Record quality check
GET	/api/harvest/quality	Processing	Quality check history
GET	/api/harvest/statistics	Manager	Harvest statistics

Production (16 endpoints)

Method	Endpoint	Access	Description
GET	/api/production/cycles	Attendant	List production cycles
POST	/api/production/cycles	Manager	Create cycle
GET	/api/production/cycles/:id	Attendant	Get cycle details

Method	Endpoint	Access	Description
PUT	/api/production/cycles/:id	Manager	Update cycle
POST	/api/production/cycles/:id/daily-log	Attendant	Add daily log
GET	/api/production/cycles/:id/logs	Attendant	Get cycle logs
GET	/api/production/cycles/:id/care-dashboard	Attendant	Care dashboard
GET	/api/production/cycles/:id/health-checks	Attendant	Health checks
POST	/api/production/cycles/:id/health-checks	Attendant	Add health check
GET	/api/production/cycles/:id/vaccinations	Attendant	Vaccinations
POST	/api/production/cycles/:id/vaccinations	Attendant	Add vaccination
GET	/api/production/cycles/:id/feed-records	Attendant	Feed records
POST	/api/production/cycles/:id/feed-records	Attendant	Add feed record
GET	/api/production/cycles/:id/environment-records	Attendant	Environment records
POST	/api/production/cycles/:id/environment-records	Attendant	Add environment record

Medications (7 endpoints)

Method	Endpoint	Access	Description
GET	/api/medications	Attendant	List medications
GET	/api/medications/active	Attendant	Active medications
GET	/api/medications/expiring	Attendant	Expiring soon
GET	/api/medications/cycle/:cycleId	Attendant	Cycle medications
POST	/api/medications	Attendant	Record medication
PUT	/api/medications/:id/complete	Attendant	Mark completed
PUT	/api/medications/:id/cancel	Attendant	Cancel medication

Employees (7 endpoints)

Method	Endpoint	Access	Description
GET	/api/employees	Manager	List employees
GET	/api/employees/departments	Manager	List departments
GET	/api/employees/stats	Manager	Employee statistics
GET	/api/employees/user/:userId	Manager	Get employee profile
POST	/api/employees	Manager	Add employee
PUT	/api/employees/:id	Manager	Update employee
PUT	/api/employees/:id/status	Manager	Toggle status

CRM (22 endpoints)

Method	Endpoint	Access	Description
GET	/api/crm/dashboard	Staff	CRM dashboard
GET	/api/crm/customers	Staff	List customers
GET	/api/crm/customers/:id	Staff	Customer details
POST	/api/crm/customers	Staff	Add customer
PUT	/api/crm/customers/:id	Staff	Update customer
GET	/api/crm/customers/:id/profile	Staff	Customer profile
POST	/api/crm/customers/:id/profile	Staff	Create profile
PUT	/api/crm/customers/:id/profile	Staff	Update profile
GET	/api/crm/loyalty	Staff	Loyalty programs
POST	/api/crm/loyalty	Manager	Create loyalty program
GET	/api/crm/loyalty/:id/enrollments	Staff	Program enrollments
POST	/api/crm/loyalty/:programId/enroll/:customerId	Staff	Enroll customer
POST	/api/crm/loyalty/:id/points	Staff	Add points
GET	/api/crm/feedback	Staff	List feedback
POST	/api/crm/feedback	Customer	Submit feedback
PUT	/api/crm/feedback/:id/status	Staff	Update status
POST	/api/crm/feedback/:id/respond	Staff	Add response
GET	/api/crm/campaigns	Staff	List campaigns
POST	/api/crm/campaigns	Manager	Create campaign
PUT	/api/crm/campaigns/:id	Manager	Update campaign
GET	/api/crm/campaigns/:id/stats	Staff	Campaign stats
GET	/api/crm/export/customers	Manager	Export customers CSV

Notifications (7 endpoints)

Method	Endpoint	Access	Description
GET	/api/notifications	Authenticated	List notifications
GET	/api/notifications/unread-count	Authenticated	Unread count
PUT	/api/notifications/:id/read	Authenticated	Mark as read
PUT	/api/notifications/read-all	Authenticated	Mark all as read
DELETE	/api/notifications/:id	Authenticated	Delete notification
GET	/api/notification-configs	Staff	List notification configs
PUT	/api/notification-configs/:id	Manager	Update config

Config, Compliance, Data, Logs, API Keys (17 endpoints)

Method	Endpoint	Access	Description
GET	/api/config	Authenticated	Get system config
PUT	/api/config	Farm Manager	Update config
GET	/api/compliance/quality	Staff	Quality checks
POST	/api/compliance/quality	Staff	Record quality check
GET	/api/compliance/audits	Staff	Audit history
POST	/api/compliance/audits	Manager	Schedule audit
GET	/api/compliance/report	Manager	Compliance report
GET	/api/data/stats	Manager	System statistics
POST	/api/data/backup	Farm Manager	Create backup
POST	/api/data/restore/:filename	Farm Manager	Restore backup
GET	/api/data/export/:type	Staff	Export data
GET	/api/system-logs	Manager	Query logs
DELETE	/api/system-logs/cleanup	Farm Manager	Cleanup old logs
GET	/api/system-logs/stats	Manager	Log statistics
GET	/api/api-keys	Farm Manager	List API keys
POST	/api/api-keys	Farm Manager	Generate key
DELETE	/api/api-keys/:id	Farm Manager	Revoke key

Analytics & Health (6 endpoints)

Method	Endpoint	Access	Description
GET	/api/analytics/production	Staff	Production analytics
GET	/api/analytics/sales	Staff	Sales analytics
GET	/api/analytics/inventory	Staff	Inventory analytics
GET	/api/analytics/p-and-l	Farm Manager	Profit & Loss
GET	/api/analytics/inventory/abc	Staff	ABC analysis
GET	/api/health	Public	System health check

Total: ~127 API endpoints

8.3 External Service Interfaces

Service	Status	Integration Point
Google OAuth 2.0	Implemented	google-auth-library for ID token verification
PayFast	Planned	Payment gateway for card payments
Yoco	Planned	Alternative payment gateway
Twilio	Planned	SMS notifications

Service	Status	Integration Point
Africa's Talking	Planned	SMS notifications (Africa)
AWS S3	Planned	Cloud storage for images/documents
Gmail SMTP	Planned	Email notifications

8.4 Deployment Interface

Component	Configuration
Backend Hosting	Render.com (Node.js web service, free tier)
Database	Render PostgreSQL (free tier)
Frontend Hosting	GitHub Pages (static)
Build Command	cd backend && npm install
Start Command	cd backend && node server.js
Health Check	GET /api/health
Auto Deploy	Enabled (from GitHub main branch)
Environment Variables	NODE_ENV, PORT, JWT_SECRET, JWT_EXPIRES_IN, DATABASE_URL

9. System Features — Detailed Specifications

9.1 Authentication Flow

1. User submits login form (email + password)
2. Route handler validates input (express-validator)
3. UserService.findByEmail() retrieves user from database
4. bcrypt.compare() verifies password hash
5. If failed: increment login attempts, check lockout threshold
6. If successful: reset login attempts, generate JWT token
7. Token returned with user object (id, email, name, role)
8. Client stores token in localStorage
9. Subsequent requests include: Authorization: Bearer <token>
10. Auth middleware verifies token, attaches req.user

Registration Flow:

1. User submits registration form
2. Only Farm Manager and Poultry Attendant roles allowed for self-registration
3. Password complexity validated (8+ chars, upper, lower, number, special)
4. bcrypt.hash() creates password hash (12 rounds)
5. User record created with status='active' (auto-approved)
6. JWT token issued and returned
7. Client stores token and redirects based on role

Password Reset Flow:

1. User requests reset via email
2. Crypto token generated (1hr expiry, single-use)
3. PasswordReset record created in database
4. (Planned) Email sent with reset link
5. User submits token + new password
6. Token validated, password updated
7. All existing tokens invalidated (optional)

9.2 Order Processing Pipeline

```
CART → CHECKOUT → ORDER CREATION
|
|— Customer adds items to cart (localStorage + server sync)
|— Checkout form: delivery option, address, payment method
|— Order created with status='pending', payment_status='pending'
|
PAYMENT PROCESSING
|
|— Staff records payment (cash/card/EFT/mobile)
|— Payment status updated to 'completed'
|— Order status updated to 'confirmed'
|
INVENTORY RESERVATION
|
|— FIFO picking algorithm selects oldest batches
|— Batch quantities reserved (status='reserved')
|— Picking list generated for warehouse staff
|
PREPARATION
|
|— Order status → 'preparing'
|— Items picked and verified
|— Order status → 'ready'
|
DELIVERY
|
|— Delivery scheduled
|— Order status → 'delivered' or 'completed'
|— Loyalty points awarded (if applicable)
```

9.3 Harvest-to-Inventory Pipeline

```
PRODUCTION CYCLE (active)
|
|— Birds reach target weight/age
|
HARVEST BATCH CREATED
|
|— Birds counted, live weight recorded
|— Harvest date and shift logged
|
PROCESSING BATCHES CREATED (per product type)
|
|— Whole Chicken, Cut-up, Portions, etc.
|
PROCESSING STEPS TRACKED
|
|— Slaughter → Pluck → Eviscerate → Cut → Portion
|— → Package → Label → Freeze → Store
|— Each step: start time, birds processed, notes
|
YIELD RECORDED
|
|— Input weight (live bird)
|— Output weight (processed product)
|— Waste weight (offal, feathers)
|— Yield % calculated
|
QUALITY CHECK
|
|— Temperature, visual, weight, hygiene, packaging
|— Result: Pass / Fail / Conditional
|
INVENTORY BATCH AUTO-CREATED
|
|— Product available for sale
|— Batch number, expiry date, storage location
|— Stock quantity updated on Product record
```

9.4 Loyalty Points System

Tier	Points Range	Discount
Bronze	0 - 999	0%
Silver	1,000 - 4,999	5%
Gold	5,000 - 9,999	10%
Platinum	10,000 - 19,999	15%
Diamond	20,000+	20%

Points Calculation:

- 1 point per ZAR 10 spent (configurable via `points_per_rand`)
- Points awarded after order completion
- Points deducted on qualifying purchases

9.5 Inventory FIFO Picking

When order is confirmed:

1. Query InventoryBatch for product, sorted by received_date ASC
2. Iterate through batches:
 - a. If batch.quantity >= order.quantity → pick from this batch
 - b. If batch.quantity < order.quantity → pick all, move to next batch
 - c. Track total picked across batches
3. Update batch quantities (subtract picked)
4. Mark batch status as 'used' if quantity reaches 0
5. Generate picking list with batch details

9.6 Notification System

Notification Types:

- Low stock alerts
- Order status updates
- Payment confirmations
- Vaccination reminders
- Medication expiry warnings
- Quality check results
- System alerts

Channels:

- In-app (implemented)
- Email (planned)
- SMS (planned)
- Push notifications (planned)

Configuration:

- Per-type enable/disable
- Frequency: immediate, daily, weekly
- Channel selection per type
- Condition-based triggers (JSON rules)

10. Appendices

Appendix A: Test User Accounts

Role	Email	Password
Admin	admin@bohlokofarm.co.za	Test@123456
Farm Manager	manager@bohlokofarm.co.za	Test@123456
Poultry Attendant	attendant@bohlokofarm.co.za	Test@123456
Processing Staff	processing@bohlokofarm.co.za	Test@123456
Sales Assistant	sales@bohlokofarm.co.za	Test@123456
Customer 1	customer1@example.com	Test@123456
Customer 2	customer2@example.com	Test@123456
Customer 3	customer3@example.com	Test@123456
Customer 4	customer4@example.com	Test@123456
Customer 5	customer5@example.com	Test@123456

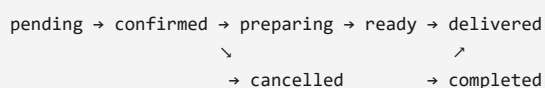
Appendix B: Product Categories

Category	Products
Live Birds	Broilers, Layers, Old Layers, Roosters
Processed	Whole Chicken, Cut-up Chicken, Portions (Breast, Thigh, Wing, Drumstick)
Value-Added	Mince, Sausages, Marinated Chicken, Ready-to-Cook
By-Products	Feet, Heads, Offal, Feathers, Manure

Appendix C: Pricing Tiers

Tier	Description	Typical Discount
Consumer	Individual buyers	Base price
Restaurant	Bulk orders, consistent pricing	5-10% off
Retailer	Wholesale with volume discounts	10-15% off
Distributor	Maximum volume, lowest price	15-20% off

Appendix D: Order Status Flow



Appendix E: Payment Methods

Method	Description	Status
Cash	On-delivery payment	Implemented

Method	Description	Status
Card	Credit/debit card (PayFast/Yoco)	Planned
EFT	Bank transfer with proof upload	Implemented
Mobile	Mobile money platforms	Implemented

Appendix F: Configuration Defaults

Setting	Default Value
VAT Rate	15%
Local Delivery Fee	R50
Free Delivery Threshold	R500
Low Stock Threshold	10 units
Currency	ZAR (R)
Session Timeout	30 minutes
Max Login Attempts	5
Lock Duration	30 minutes
JWT Expiry	24 hours
Rate Limit	100 req/15min
bcrypt Rounds	12
Backup Retention	30 days
Log Retention	90 days

Appendix G: Glossary

Term	Definition
Batch	A group of birds or products tracked together through the system
Cycle	A production cycle from bird procurement to harvest
Dressing Percentage	$(\text{Carcass weight} / \text{Live weight}) \times 100$
FIFO	First In, First Out — oldest inventory used first
Harvest	The process of slaughtering and collecting birds for processing
Lot	A traceable group of products from the same production source
Mortality Rate	$(\text{Deaths} / \text{Starting count}) \times 100$
Processing	Converting live birds into marketable products
Withdrawal Period	Time after medication before bird can be processed for food

Document End

Generated from comprehensive analysis of 48 markdown documents, 36 PDF documents, 100+ code files, and 15+ HTML pages in the BohlokoFamilyFarm repository.